

Detailed Design

TP3 discussion-board staff-support extension for FoundationsS26 *Use-case, CRC, class, sequence, activity, and state design with implementation outline*

Purpose. This document explains how the TP3 functionality will be realized in Java by refining the existing Foundations/TP2 codebase into a staff-capable discussion and assessment system. The focus is on the new staff-facing user stories: invitation and account control, role-sensitive access, report review, discussion moderation, and participation-analysis support.

1. Detailed Design Scope and Functional Objectives

- Preserve the existing layered Foundations structure: JavaFX view/controller pages, entity classes, and H2-backed Database support.
- Realize the new staff user stories through focused enhancements to existing packages rather than a wholesale rewrite.
- Support realistic staff workflows: invite users, activate/deactivate accounts, adjust roles, reset passwords, review posts and replies, inspect reports, and assess participation coverage.
- Keep the design traceable to Java implementation by centering the model around actual classes already visible in the project branch, including Database, AdminServiceImpl, Post, Reply, ReportInfo, ViewDiscussionBoard, and AdminUsersDialog.

2. Selected Detailed Design Models

The following models were selected because they best explain how the new functionality is realized using Java in this codebase:

- Use-Case ‘Diagram’ view: shows actors and the highest-value staff functions.
- Use-Case Scenarios: turns each staff function into a concrete interaction path.
- CRC Cards: clarifies which classes own which responsibilities and collaborations.
- Class ‘Diagram’ view: captures static structure and dependencies across packages.
- Sequence-style interaction summaries: explains runtime message flow for key operations.
- Activity/state-style models: captures workflow and object lifecycle decisions that are harder to show in the class ‘diagram’ alone.

3. Use-Case Diagram (Textual UML View)

Primary actors and use cases are summarized below. This section stands in for the visual use-case diagram and uses the same actor-to-use-case mapping that would appear in UML.

Actor	Primary Use Cases	Notes / System Boundary
Administrator / Staff Reviewer	Send invitation; list/filter users; activate/deactivate user; change user role; reset password; open discussion board; review reports; inspect abusive content; review participation evidence	Main TP3 actor. Combines administrative control with assessment support.
Role1 User / Student-like participant	Open discussion board; create post; edit own post; delete own post; reply to post; edit own reply; report abuse; update own account	Continues TP2 student-side behavior; generates discussion data used by staff.
Role2 User / Reviewer-like participant	Open discussion board; participate in discussion; report abuse; update own account	Alternative non-admin actor with discussion access.
System / Database	Persist users, invitations, posts, replies, solved state, reports, and role/account state	Acts as the authoritative store and policy enforcement point for multiple operations.

4. Representative Use-Case Scenarios

UC-STAFF-01 Review user accounts and update status

- Preconditions: staff user is authenticated and routed to the Admin home page.
- Main flow: the actor opens AdminUsersDialog, optionally filters by role or active status, selects a user, then activates/deactivates the account or changes the role.
- Postconditions: the selected user record reflects the new status or role, and the refreshed list shows the change.
- Alternate flows: operation is rejected if the actor lacks authorization, if the target user does not exist, or if the change would violate protections such as preserving the last active administrator.

UC-STAFF-02 Reset a user password

- Preconditions: staff user is authenticated as an administrator.
- Main flow: the actor selects a user in AdminUsersDialog and requests a one-time password reset. AdminServiceImpl generates a temporary password and Database updates the target account and password-reset flag.
- Postconditions: the temporary password is shown to the authorized actor, the target account is marked to reset on next use, and the action is auditable through the service/database path.
- Alternate flows: unauthorized actor or invalid target user causes the operation to fail without changing the account.

UC-STAFF-03 Review reports and “abusive” content

- Preconditions: reports exist in the reports table and staff opens the report-review path from Admin home.
- Main flow: ViewDiscussionBoard.displayReports shows the reports table, the actor selects a report, inspects the underlying post or reply, and decides whether further action is needed.
- Postconditions: the staff member has enough information to evaluate the report and, if desired, remove the report entry after review.
- Alternate flows: no reports exist, so the report table displays an empty placeholder instead of actionable entries.

UC-STAFF-04 Assess participation threshold

- Preconditions: posts and replies exist for a discussion topic or evaluation window.
- Main flow: a coverage-analysis component receives discussion questions and replies, groups answers by responding student, collapses duplicate answers to the same peer, ignores self-answers, and determines whether the student met the threshold of answering at least three different peers.
- Postconditions: staff receives a summary that identifies compliant students, students below threshold, unanswered questions, and orphan reply anomalies.
- Alternate flows: invalid thresholds or incomplete data are rejected or flagged so the staff member does not rely on misleading summaries.

UC-STUDENT-01 Create and manage discussion content

- Preconditions: non-admin participant is authenticated and has discussion-board access through Role1 or Role2 home page.
- Main flow: the actor opens the discussion board, creates a post or reply, edits or deletes owned content, marks or reopens a discussion if permitted, and reports abusive content when necessary.
- Postconditions: discussion content and metadata are persisted and immediately visible to future readers and staff reviewers.
- Alternate flows: the request is denied when content violates validation rules, is too similar to existing content, uses inappropriate language, or the actor tries to modify content owned by someone else.

5. CRC Cards

The CRC cards below concentrate on the classes that realize the new TP3 staff behavior and the discussion-board flows reused from TP2.

Class	Responsibilities	Collaborators
ViewAdminHome	Displays admin status information and launches invitation, user-management, discussion-board, and report-review actions.	ControllerAdminHome; Database; AdminUsersDialog; ViewDiscussionBoard; ViewUserUpdate
ControllerAdminHome	Validates invitation requests, dispatches admin actions, and routes the user into staff workflows.	ViewAdminHome; Database; AdminUsersDialog; ViewDiscussionBoard
AdminUsersDialog	Presents filterable user list; gathers staff commands for activate/deactivate, role change, and password reset.	AdminService; AdminServiceImpl; UserSummary; Role
AdminServiceImpl	Implements authorization-aware admin operations and protects critical invariants such as the last active admin.	Database; UserSummary; Role; PasswordUtil
ViewDiscussionBoard	Displays posts, replies, report list, and moderation actions; delegates business decisions to controller.	ControllerDiscussionBoard; Post; Reply; ReportInfo
ControllerDiscussionBoard	Loads posts/replies/reports; validates create/edit/delete/report actions; enforces ownership/admin checks; coordinates persistence.	Database; Post; Reply; ReportInfo; contentLanguageChecker
Database	Persists user, invitation, post, reply, and report data; supplies query/update methods; acts as system-of-record.	User; Post; Reply; ReportInfo; CreateInfo; Role
Post / Reply	Represent discussion content, edit metadata, solved state, summary/body access, and shared interface for reporting.	CreateInfo; DiscussionBoardContent; Database; ViewDiscussionBoard
StudentCoverageAnalyzer and result types	Evaluate staff-facing participation summaries such as distinct-peer count and threshold compliance.	Post; Reply; StudentCoverageResult; StudentCoverageSummary

6. Class Diagram View (Textual UML Summary)

The detailed design is implemented as a layered package structure. The class-diagram narrative below captures the major classes, inheritance/interface points, and associations that matter to TP3.

Package / Layer	Class or Interface	Key Relationships	Design Intent
applicationMain	FoundationsMain	Owns shared Database reference; dispatches startup flow to FirstAdmin or UserLogin	Single application entry and coarse-grained flow control.
entityClasses	DiscussionBoardContent	Implemented by Post and Reply	Enables shared handling of reported content and summaries.
entityClasses	CreateInfo	Composed inside Post and Reply	Centralizes creation/edit timestamps and content identifiers.
entityClasses	Post	Composes CreateInfo; one-to-many conceptual relationship to Reply via postId	Represents top-level discussion questions or prompts.
entityClasses	Reply	Composes CreateInfo; references parent post by postId	Represents response content under a post.
entityClasses	ReportInfo	Wraps a DiscussionBoardContent object and report count	Allows staff review to treat post and reply reports uniformly.
database	Database	Stores/retrieves User, Post, Reply, ReportInfo; exposes authorization-sensitive update methods	Separates persistence and policy checks from UI code.
admin	AdminService	Implemented by AdminServiceImpl	Stable service boundary for staff operations.
admin	AdminServiceImpl	Depends on Database and PasswordUtil; returns UserSummary data	Encapsulates admin business rules and security checks.
guiAdminHome	ViewAdminHome / ControllerAdminHome	Calls AdminUsersDialog and ViewDiscussionBoard	Realizes staff/admin interaction points from the GUI side.
guiDiscussionBoard	ViewDiscussionBoard / ControllerDiscussionBoard	Consumes Post, Reply, ReportInfo and Database methods	Realizes discussion interaction and moderation/review flows.
analytics	StudentCoverageAnalyzer family	Consumes Post and Reply collections	Produces staff-facing assessment summaries for participation review.

7. Sequence-Style Interaction Models

SEQ-01 Staff password reset

- Actor -> AdminUsersDialog: select user and click Reset Password
- AdminUsersDialog -> AdminServiceImpl: resetPassword(...) or resetPasswordAs(...)
- AdminServiceImpl -> PasswordUtil: generateStrongTemp(length)
- AdminServiceImpl -> Database: updatePasswordAuthorized(target, actor, newPassword)
- AdminServiceImpl -> Database: updateUserPasswordReset(target, true)
- AdminServiceImpl -> AdminUsersDialog: return temporary password / success status
- AdminUsersDialog -> Actor: display result

SEQ-02 Staff reviews report

- Actor -> ViewAdminHome: choose Review Reports
- ViewAdminHome -> ViewDiscussionBoard: displayReports(stage)
- ViewDiscussionBoard -> ControllerDiscussionBoard: load reports and report headers
- ControllerDiscussionBoard -> Database: getAllReports(); getContentById(...) as needed
- Actor -> ViewDiscussionBoard: select report and inspect content
- ViewDiscussionBoard -> ControllerDiscussionBoard: retrieve related post/reply details
- ControllerDiscussionBoard -> Database: fetch report target and related parent content
- System -> Actor: show reviewed content and context

SEQ-03 Student creates post

- Actor -> ViewDiscussionBoard: click Create
- ViewDiscussionBoard -> ControllerDiscussionBoard: createPost()
- ControllerDiscussionBoard -> validation helpers: content presence, length, language, spam/similarity checks
- ControllerDiscussionBoard -> Database: createPost(new Post(...))
- Database -> ControllerDiscussionBoard: persist row in posts table
- ControllerDiscussionBoard -> ViewDiscussionBoard: refresh posts list

SEQ-04 Staff participation analysis

- Actor -> analytics component: request participation summary for threshold 3
- StudentCoverageAnalyzer -> Post/Reply collections: group posts and replies by owner and target peer
- StudentCoverageAnalyzer -> StudentCoverageSummary: compute unique-peer counts and remaining peers
- StudentCoverageAnalyzer -> StudentCoverageResult: package compliant, non-compliant, orphan, and unanswered findings
- System -> Actor: display coverage summary suitable for grading review

8. Activity and State Models

8.1 Activity Model: Invitation to active user

- Start -> staff enters email and selects target role.
- Validate email format and ensure invitation has not already been sent.
- Generate invitation code and persist invitation row.
- Potential user enters invitation code and reaches new-account page.
- Validate username, validate password, ensure passwords match, and ensure username is unique.
- Create user account with the invited role and continue into account-update / role-dispatch flow.

8.2 Activity Model: Report review and moderation

- Start -> staff opens report list.
- Filter/select report target.
- Fetch associated post/reply and surrounding context.
- Inspect content, report count, and parent discussion context.
- Decide: keep for later review, dismiss/remove report, or take follow-on account/content action.

8.3 State Model: Post lifecycle

State	Valid Transitions	Meaning
Draft (dialog only)	Created -> Published; Cancelled -> End	Temporary UI state before persistence.
Published / Unsolved	Edit; Delete; Reply Added; Mark Resolved; Reported	Normal active discussion state.
Published / Solved	Re-Open; Delete; Reported	Discussion considered resolved but still reviewable.
Reported	Review by staff; report count increment; return to prior visible state	Overlay condition indicating moderation attention.
Deleted	No outgoing transitions	Terminal persistence state after removal.

9. Multilevel Outline: How the Detailed Design Enables the Java Implementation

9.1 Functional objectives

- 9.1.1 Invitation and onboarding are realized through ViewAdminHome, ControllerAdminHome, ViewNewAccount, and ControllerNewAccount, with Database providing invitation lookup and account creation support.
- 9.1.2 Staff account control is realized through AdminUsersDialog plus AdminServiceImpl and Database authorization methods for status changes, role changes, and password reset.
- 9.1.3 Staff discussion review is realized through ViewDiscussionBoard report mode, ControllerDiscussionBoard, ReportInfo, and Database report/content queries.
- 9.1.4 Student discussion participation is realized through the same discussion-board package using role-sensitive actions and persistence methods for posts and replies.
- 9.1.5 Participation analysis is realized through analytics classes that consume persisted discussion data and produce staff-facing summary results.

9.2 Quality objectives

- 9.2.1 Maintainability is improved by keeping new staff logic close to existing page/package boundaries and by centralizing repetitive admin rules in AdminServiceImpl.
- 9.2.2 Traceability is improved because the design maps directly to real Java packages, controller actions, service calls, and entity/database methods already present in the codebase.
- 9.2.3 Usability is improved by separating staff review views from ordinary participant pages and by using filtered tables, dialogs, and explicit review actions.
- 9.2.4 Security and integrity are improved by putting role-sensitive operations behind service/database authorization methods rather than allowing arbitrary GUI-side mutation.

9.3 Reuse objectives

- 9.3.1 Existing Foundations dispatch, login, update-account, and JavaFX page patterns are reused rather than replaced.
- 9.3.2 The entity layer is reused through shared content abstractions such as DiscussionBoardContent and CreateInfo.
- 9.3.3 The same Database object and H2 persistence layer are reused for both student and staff workflows, limiting architectural churn.
- 9.3.4 The analytics path reuses Post and Reply instead of inventing a separate grading-only content representation.

9.4 Java realization decisions

- 9.4.1 Views own widget creation and event wiring; controllers own action decisions; services own cross-cutting staff rules; Database owns persistence.
- 9.4.2 Shared IDs and metadata from CreateInfo allow lookup, reporting, and relationship traversal without introducing a second identity scheme.
- 9.4.3 Staff features are implemented as incremental enhancements-new dialogs, review tables, authorization helpers, analytics classes-so the team can develop from TP2 into TP3 without destabilizing the existing application.

9.5 Iteration guidance

- 9.5.1 If coding confusion appears, the team should trace backward from the failing user story to the relevant use case, CRC card, class relationship, and sequence path.
- 9.5.2 If the code changes enough to alter package boundaries or ownership of responsibilities, the class-diagram and CRC sections must be updated before final submission.
- 9.5.3 If testing reveals defects in authorization, ownership checks, or threshold analysis, the affected sequence/activity/state sections must be revised so the documentation still matches the code.

10. Design Notes for Final Alignment

- This detailed design intentionally favors models that map cleanly to the current Java classes so the team can use the document while coding, testing, and recording screencasts.
- Where the implementation still contains defects or transitional naming, those should be corrected in code and then reflected back into this design before final PDF export.
- The document should be read together with Architecture.pdf, Staff User Stories.pdf, New Code Javadoc, Test Code Javadoc, and Manual Tests.pdf so that requirements, design, implementation, and validation flow as one coherent set.